CECS 327 Sec 01 Distributed File System 2 Report

Group Members:

Dean Vu

Anthony Phan

Evan Gonzalez

Chord Integration

This project extends a Chord distributed hash table into a simplified distributed file system. Each node is assigned a node ID using SHA-1 hashing, and keys are mapped into the same identifier space. The ring size used is (2^8 = 256).

When nodes join the system, they are sorted by node ID to form the Chord ring. Each node also builds a finger table that stores successor information for efficient lookup. The function locate_successor() is used to find the node responsible for a given key.

The demo creates five peers on ports 8000–8004 and prints the node IDs and finger tables at startup.

Metadata and Page design

Our distributed file system stores files using a metadata object and a collection of fixed size pages. Metadata is stored separately from the file contents so that the system can track information about the file like organization, page locations and version information without retrieving the full file data. When a file is appended into our distributed file system, the records are divided into pages and distributed across the

Chord ring. During reads, the DFS retrieves the pages in order and combines them to remake the original file contents.

Distributed Sorting Strategy

The project includes a sorting demo using a DFS file containing 130 records.

The sorting process:

1. Reads all pages from the input DFS file
2. Combines the records
3. Sorts the records globally
4. Writes the sorted records into a new DFS output file

The output file is stored using the same page structure as any other DFS file. Correctness is verified using the function verify_sorted(), which checks whether the output matches Python's sorted version of the records. The demo prints:

Globally sorted: True

The first and last ten records are also printed for verification.

Replication Strategy

The system uses successor-based replication with replication factor:

R = 3

Metadata and pages are replicated on the responsible Chord node and the next two successors in the ring.

Replication improves fault tolerance because data can still be retrieved if one replica crashes. During reads, the DFS attempts to retrieve data from the first available replica.

The demo prints replica locations for both metadata and pages.

Paxos message Flow

The project uses a simplified Paxos protocol to keep replicated metadata consistent across replicas. Each file within our Distributed file System has a Paxos group containing three replicas and one leader node. When an update occurs, the leader creates a new ballot number and sends an ACCEPT message to all replicas. Once a majority of the replicas respond with a LEARN message, the operation is committed and applied to all replicas in the same order. Our system is also designed to operate even if a node fails, since Paxos can still reach a majority.

Failure Assumptions

The project assumes crash-stop failures only and does not handle Byzantine failures. Messages can be delayed, lost, duplicated, or reordered.

The demo includes:

- delayed Paxos messages
- replica crash during replication

Even after one replica crashes, the operation still commits because two replicas form a majority.

Limitations and Future Improvements

Limitations:

- The system uses a simplified version of Paxos
- The system runs the process on a single machine rather than multiple distributed machines
- System wouldn't handle very large files as efficiently as other systems

Future Improvements:

- Add real network communication between peer to peer
- Implement a more in depth version of the Paxos protocol
- Support a larger Chord rings with more peers involved
- Improve distributed sorting performance to handle larger files

Test Evidence

(all located in the zip file submission)

Sample input file: (a.txt, b.txt, c.txt, unsorted_130.txt)

Sorted output file:(sorted_out.csv)

Paxos messages: (demo_logs.txt)

Correctness check: Our DFS layer has a built-in verify_sorted() function. The result of this check will print directly to the demo_logs.txt file in the zipped folder. The line "Globally sorted: True" will show that the algorithm worked correctly.